FEAT-ZD-48600 · DE-06 Data Mapping · Updated data mappings and transformation steps

Data Engineering agent · item feat.mappings · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/feat.mappings.md · approved by Data engineer sim.data_engineer

feat.mappings: FEAT-ZD-48600

Updated data mappings and transformation steps for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-06, Data Mapping, attempt 2. The workbook action is "Update the data mapping based on the data model and fetch the latest data mappings based on the customer input". There is no gate. Automated checks, the independent judge and sampled human evaluation review this output (record:brief).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-06 Data Mapping, attempt 2	record:brief
Inputs	feat.data_model_change (commit 7b68a18, from DE-05), org.customer_mappings (commit 3b03f15, feature org-override)	record:brief
Code read	ap_pipeline source, tests and `schedule.json` from the step's tool view; `data/sample_feed.csv` read from the checkout	tool:repo; file:ap-pipeline-sample/data/sample_feed.csv
Customer mapping applied	Vendor feed **v3**: `invoice_id, vendor, due_date, amount, paid, paid_date`. `paid_date` is optional, an ISO date, and empty when unpaid	org.customer_mappings
Result	**The customer contract adds one mapping,** `paid_date -> Invoice.paid_date`**. The existing column mappings (**`invoice_id`**,** `vendor`**,** `due_date`**,** `amount`**,** `paid`**) are unchanged. The mapping agrees with the Data Model change field by field. The schedule stays daily 02:00 Asia/Kolkata, which matches** `schedule.json`**.** There is no new source, no new target entity and no storage change	org.customer_mappings; feat.data_model_change (2.1, 2.2, 3); tool:repo (`schedule.json`, `transform.py`)

1. Latest customer mapping contract compared with the Data Model change

Item	Customer contract (org.customer_mappings)	Data Model change (feat.data_model_change)	Agreement	Source
Feed version	v3	v2 today; v3 adds the optional `paid_date` column	Agrees	org.customer_mappings; feat.data_model_change (2.1)
Columns	`invoice_id, vendor, due_date, amount, paid, paid_date`	The existing columns, unchanged, plus `paid_date`	Agrees	org.customer_mappings; feat.data_model_change (2.1)
`paid_date` spelling	`paid_date`	Assumed `paid_date`, open as DMQ-2 (carried CQ-I1)	Agrees. The customer contract confirms the spelling, so DMQ-2 can close (MQ-1)	org.customer_mappings; feat.data_model_change (6)

Item	Customer contract (org.customer_mappings)	Data Model change (feat.data_model_change)	Agreement	Source
`paid_date` format	Optional, ISO date, empty when unpaid	ISO date string, or empty when unpaid. Missing, `None`, blank or whitespace only gives `None`	Agrees	org.customer_mappings; feat.data_model_change (2.1, 2.2)
Target	`paid_date -> Invoice.paid_date`	Invoice `paid_date`: date, nullable (`None`); already registered	Agrees	org.customer_mappings; feat.data_model_change (2.2)
Schedule	Daily 02:00 Asia/Kolkata (unchanged)	Schedule and orchestration unchanged	Agrees, and matches `schedule.json` (section 5)	org.customer_mappings; feat.data_model_change (3); tool:repo (`schedule.json`)

2. Source-to-target mapping: feed row to Invoice

Source: a vendor invoice feed row (CSV with a header row), which `ingest.parse_feed` turns into a `dict` keyed by header text. Target: the Invoice record (a plain `dict`) that `transform.to_model` returns (tool:repo `ingest.py`, `transform.py`; feat.data_model_change 2.1, 2.2).

Source column	Target field	Target type	Nullable	Transformation rule	Change	Source
`invoice_id`	`id`	str	No	`strip()`	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`vendor`	`vendor`	str	No	`strip()`	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`due_date`	`due_date`	date	No	`date.fromisoformat(...)`; no `strip()` is added	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`amount`	`amount`	Decimal, 2 dp	No	`Decimal(...)`, never float	None	tool:repo (`transform.py`); feat.data_model_change (2.2); record:memory (D-2026-044, L-011)
`paid`	`paid`	bool	No	`strip().lower() == "true"`; a missing column means false	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`paid_date`	`paid_date`	date	**Yes (`None`)**	Missing key (feed v2), `None` (short v3 row), blank or whitespace only gives `None`. Otherwise `date.fromisoformat(value.strip())`. A non-blank invalid value rejects the row with reason `paid_date`	**New mapping** (implemented with IN-3)	org.customer_mappings; feat.data_model_change (2.1, 2.2, 2.3)

3. Transformation steps (per feed row, in this order)

The order keeps today's behaviour for the existing fields and maps `paid_date` last, so an existing reject reason wins over `paid_date` (feat.data_model_change 2.2, DD-6 / DMQ-5).

1. **Parse** the CSV text into header-keyed rows. No code change is needed for v3, because `parse_feed` keys rows by header text (tool:repo `ingest.py`; feat.data_model_change 2.1).
2. **Map the existing fields** `id`, `vendor`, `due_date`, `amount` and `paid` exactly as today. A missing or invalid field rejects the row with the exception type name as `reason` (`KeyError`, `ValueError`, `InvalidOperation`, `AttributeError`) (tool:repo `transform.py`; feat.data_model_change 2.3).
3. **Map** `paid_date` only when step 2 succeeded. Read `row.get("paid_date")`. If it is `None`, blank or whitespace only, set `paid_date = None`. Otherwise parse the stripped value as an ISO date (org.customer_mappings; feat.data_model_change 2.2).
4. **Reject an invalid** `paid_date`**.** If the parse in step 3 fails, append `{"row": row, "reason": "paid_date"}` (constant `PAID_DATE_REASON`) to the rejected list and do not load the record (feat.data_model_change 2.3).
5. **Emit the record** with exactly the keys `id`, `vendor`, `due_date`, `amount`, `paid` and `paid_date`. `paid_date` is present on every accepted record, including feed v2 rows (feat.data_model_change 2.2, 4).
6. **Keep the record-level rules.** A paid record with `paid_date = None` is kept. An unpaid record with a valid `paid_date` is kept, with the date (interim rule DD-7, DMQ-3). A `paid_date` earlier than `due_date` is kept (feat.data_model_change 2.2).
7. **Data-quality report**: no code change. `reject_reasons` is the sorted set of rejected `reason` values, so `paid_date` appears in it automatically (tool:repo `dq.py`; feat.data_model_change 2.3).
8. **Derive the Delay history** with the new pure function `delays.delays_by_vendor(records)` (section 4; feat.data_model_change 2.4).

`to_model` keeps its signature and its `(records, rejected)` return shape, and `pipeline.run` is unchanged (feat.data_model_change 3; tool:repo `pipeline.py`).

4. Derived mapping: Invoice to Delay history

Source (Invoice)	Target	Rule	Source
`vendor`	Key of `Dict[str, List[int]]`	Vendors in first-seen order. A vendor with no qualifying record has no key	feat.data_model_change (2.4)
`paid_date`, `due_date`	A `delay_days` value in the list	`(paid_date - due_date).days`, in calendar days, as an int that may be negative. Values in feed order	feat.data_model_change (2.4); record:memory (D-2026-031)
`paid`, `paid_date`	Filter	Only records where `paid` is truthy **and** `paid_date` is not `None`	feat.data_model_change (2.4)
n/a	Persistence	None: in memory only, and the record list is not modified	feat.data_model_change (2.4)
n/a	Consumer	ap_forecast `vendor_risk`, which takes one vendor's list as `history`	feat.data_model_change (2.4, 2.5)

The Invoice consumer ap_aging (`overdue_with_risk`) reads `id`, `vendor`, `due_date`, `amount` and `paid` and ignores `paid_date`, so the new mapping does not affect it (feat.data_model_change 2.5, 3).

5. Schedule mapping

Attribute	Customer contract	Pipeline config today	Change	Source
Frequency	Daily	`"frequency": "daily"`	None	org.customer_mappings ; tool:repo (`schedule.json`)
Time	02:00	`"time": "02:00"`	None	org.customer_mappings ; tool:repo (`schedule.json`)
Timezone	Asia/Kolkata	`"timezone": "Asia/Kolkata"`	None	org.customer_mappings ; tool:repo (`schedule.json`)

This step changes no schedule. It confirms that the customer frequency and the config agree, for use in DE-13 Scheduling the pipelines (org.customer_mappings; tool:repo `schedule.json`).

6. Mapping validation rules (for DE-08 Coding and DE-09 Unit Test Cases)

Case (synthetic rows only)	Expected	Source
v2 header (no `paid_date` column), valid row	Loaded; `paid_date = None`	feat.data_model_change (3, 4)
v3 row with `paid_date` empty (unpaid, as the customer sends it)	Loaded; `paid_date = None`	org.customer_mappings; feat.data_model_change (4)
v3 row with a valid ISO `paid_date`	Loaded; `paid_date` is a `date`	org.customer_mappings; feat.data_model_change (4)
v3 row with a non-blank invalid `paid_date` (`"2026-13-01"` or `"not-a-date"`)	Rejected with reason `paid_date`, which `reject_reasons` lists	feat.data_model_change (4)
Bad `due_date` and bad `paid_date` together	Rejected with reason `ValueError`	feat.data_model_change (4)
Unpaid row with a valid `paid_date`	Accepted; the date is kept	feat.data_model_change (4)
Every accepted record	Keys exactly `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`; `amount` is `Decimal`	feat.data_model_change (4); record:memory (D-2026-044)

The repository's sample feed still has the v2 header (`invoice_id,vendor,due_date,amount,paid`), so it exercises the v2 path only. v3 cases need synthetic rows in the tests (file:ap-pipeline-sample/data/sample_feed.csv).

7. Code in this step

This step updates the mapping document only and changes no code. The tool view shows `transform.py` without `paid_date` and no `delays.py` module (tool:repo). IN-3 (the `paid_date` mapping and reject reason) and IN-7 (`delays_by_vendor`) implement the mapping in DE-08 Coding (feat.data_model_change 2.2, 2.4, 5).

8. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
MQ-1	DMQ-2 (carried CQ-I1: is the v3 column spelled exactly `paid_date`?) is answered by the customer contract, which lists `paid_date`. Can DMQ-2 be closed?	Yes; this mapping uses `paid_date`	Data engineering lead / feed owner

ID	Question	Interim answer	Owner (proposed)
MQ-2	The customer contract says `paid_date` is "empty when unpaid", while the model keeps an unpaid record that carries a valid `paid_date` (DD-7, DMQ-3, interim). Should such a row be rejected, or its `paid_date` cleared, instead?	Keep it, as the model says. The Delay history ignores it because it filters on `paid`	Product / Architect; feed owner
MQ-3	The customer contract names only "v3" and gives no effective date. Must the pipeline accept v2 and v3 feeds side by side for a period?	Yes. The mapping already accepts both, because a missing column gives `None`	Data engineering lead / feed owner
MQ-4	Carried DMQ-1: should org.data_model register the `paid_date` reject reason and the Delay history structure?	Treat feat.data_model_change and this mapping as the feature-level record	Data engineering lead; data modelers

9. Validation check

Check	Result	Source
Mapping updated from the data model	Pass: every Invoice field in feat.data_model_change 2.2 has one source column (section 2), and the Delay history is mapped (section 4)	feat.data_model_change
Latest customer mapping applied	Pass: v3 columns and `paid_date -> Invoice.paid_date` (sections 1 and 2); schedule confirmed (section 5)	org.customer_mappings
Conflict with the Data Model change or the customer contract	None. One wording difference is raised as MQ-2	org.customer_mappings; feat.data_model_change
Production or source-system data read	None. Only the inputs, the tool view, repository files and synthetic rows were used	record:brief
Blocking questions	None	n/a